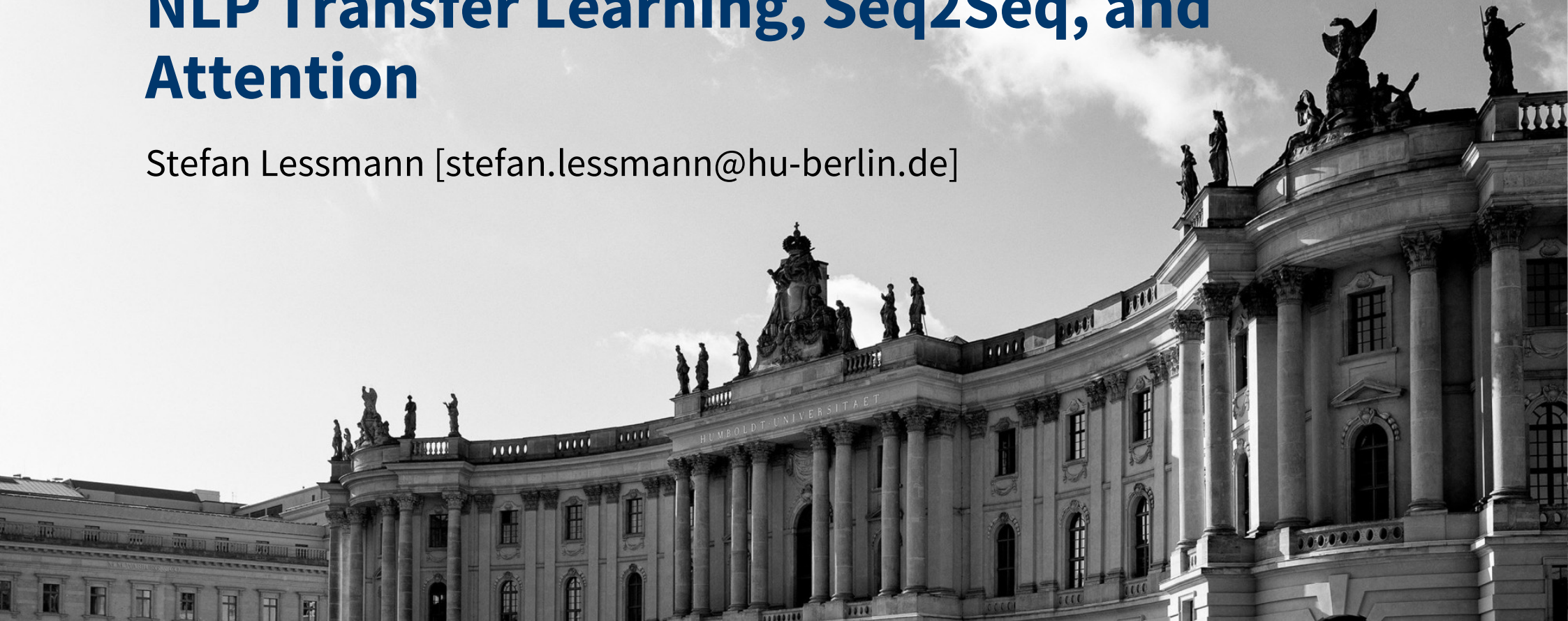


Deep Learning for Text Analytics

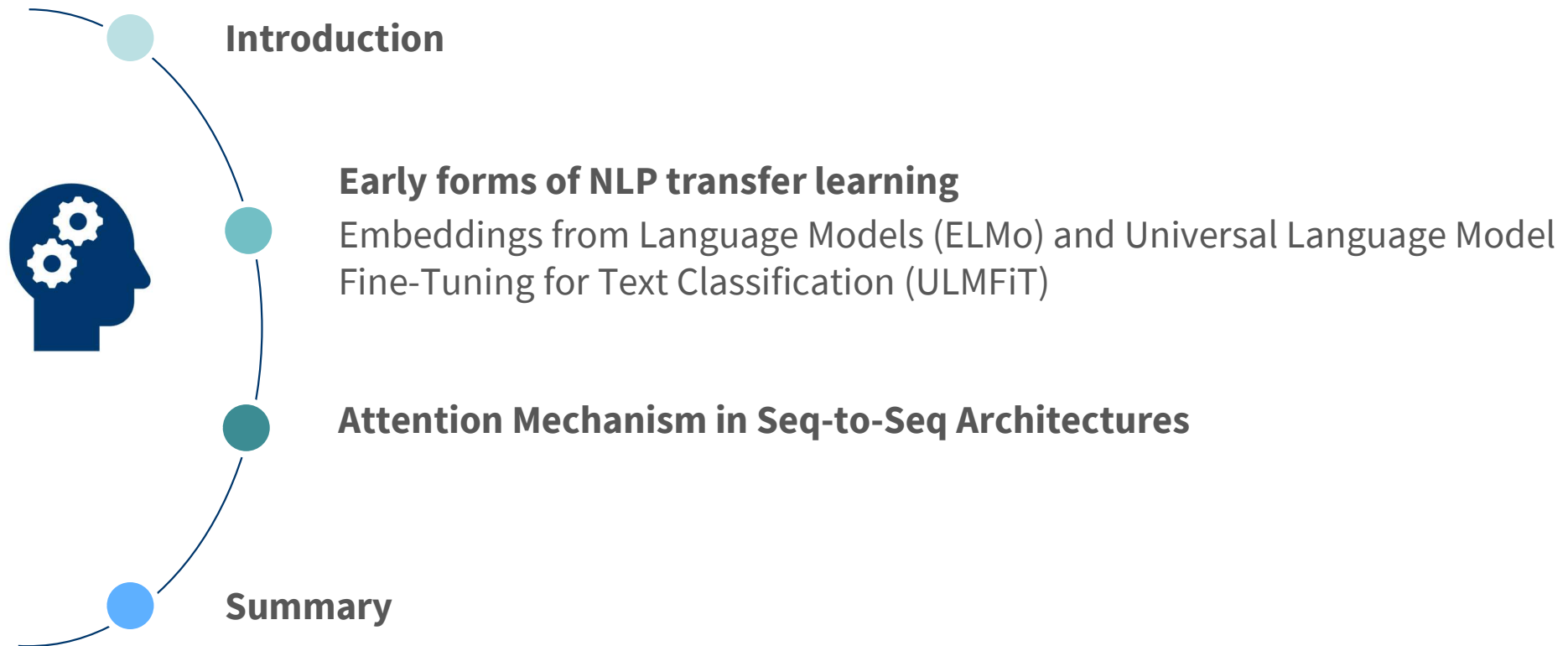
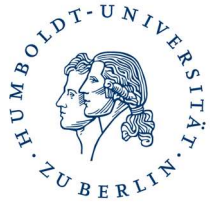


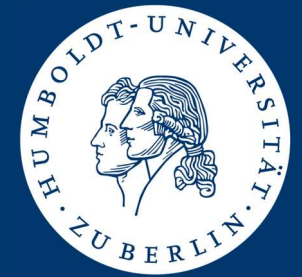
NLP Transfer Learning, Seq2Seq, and Attention

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



Agenda





Introduction

RECAP: Status-Quo with NLP and RNNs

Stacked, bidirectional RNN-type networks & embeddings

- **Embeddings to represent tokens**
- **LSTM or GRU cells to address gradient vanishing**
- **Bidirectionality to capture left & right context**
- **Stacking layers to learn more abstract patterns**
- **Self-training on large corpora**
 - Window-wise parsing of documents to obtain sequences
 - Using next token prediction as objective (i.e., LM)
 - Train weights using cross-entropy loss by backprop.

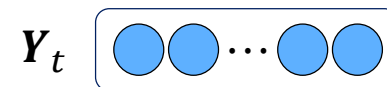


Corpus

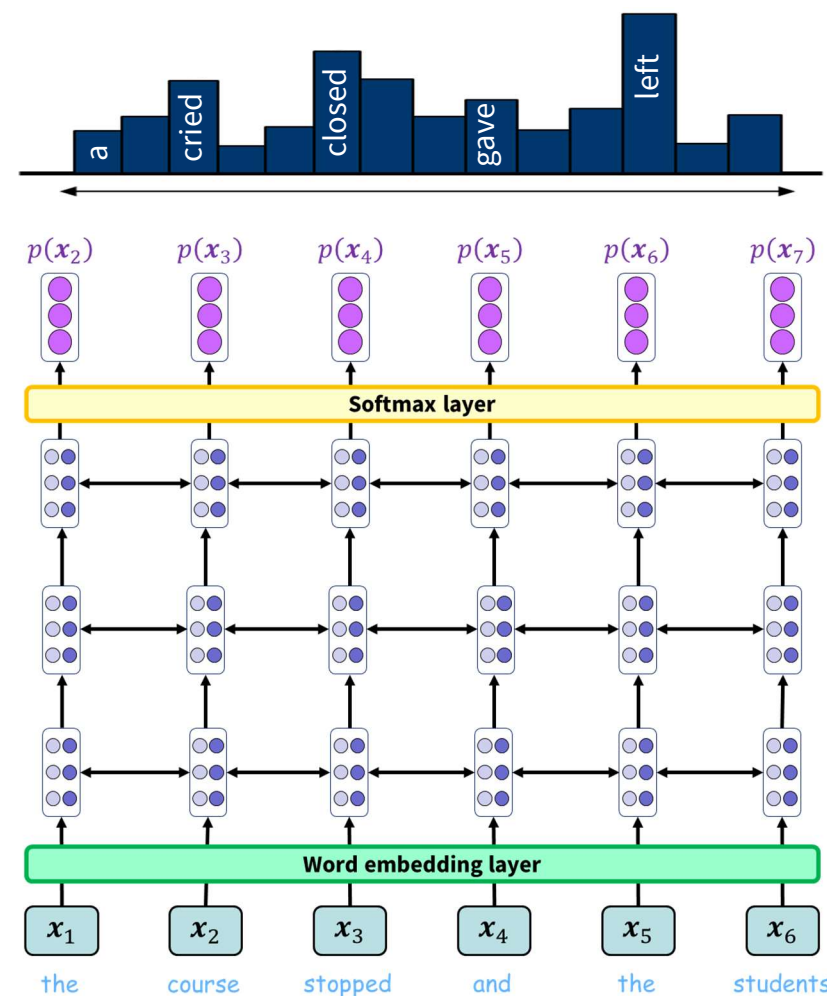


Parsing

started as early as eight in
in the morning <comma> talking about
about RNNs <dot> After a daunting
daunting two hours it eventually turned
eventually turned ten and the course
the course stopped and the students
students closed their books and left



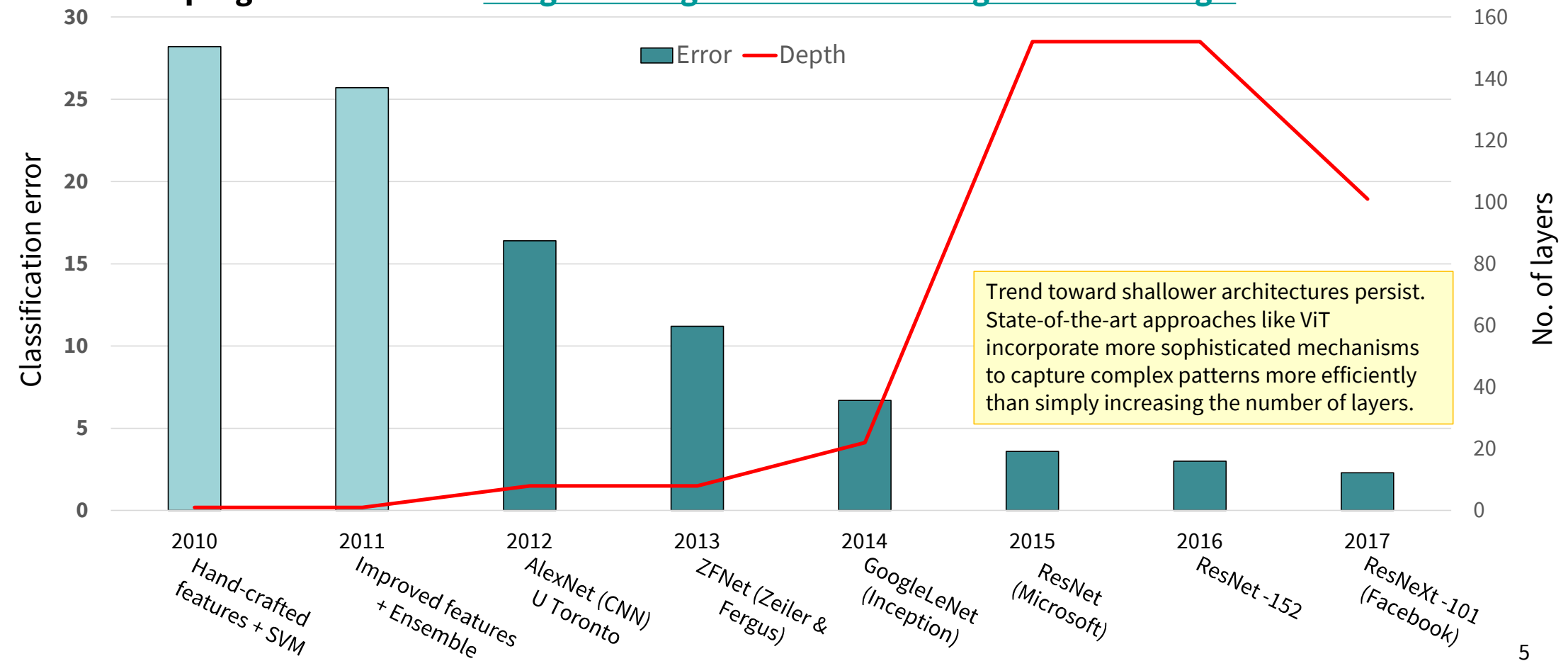
$$\min_W J(Y_t, \hat{P}_t)$$



Quo Vadis NLP?

Mimic practices from computer vision (CV), where progress was faster

CV progress in terms of [ImageNet Large Scale Visual Recognition Challenge](#) results



NLP Transfer Learning (aka “ImageNet Moment”)

Beyond shallow W2V embeddings

■ Deep pre-trained ImageNet models used successfully in many applications

- Earlier layers learn more basic patterns
- Deep learners learn higher-level abstractions

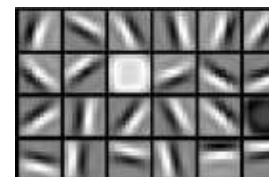
■ For NLP, word vectors facilitate pre-training first-layer weights (i.e. shallow)

■ How pretrain entire models with complex representations of language?

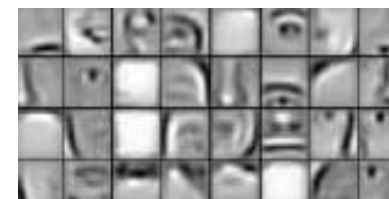
- Requires a task that encourages the learning of features that **generalize** to new NLP tasks
- Requires a task where a **large** set of data is available for pretraining



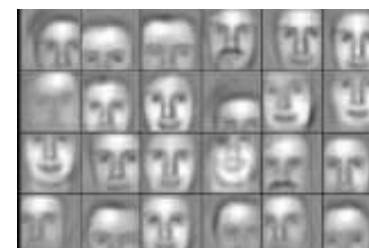
Pixels



Edges

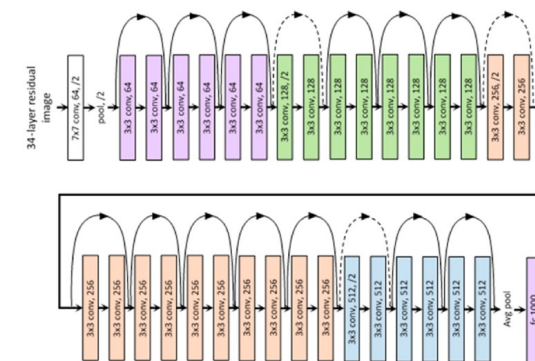


Object parts
(combinations
of edges)



Object
models

ResNet architecture

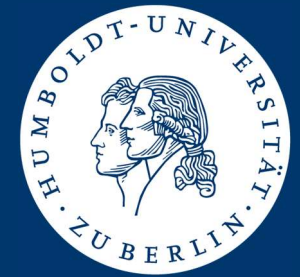


[Image source](#)

NLP Transfer Learning

Language modeling as the task for NLP transfer Learning

- **Language models need syntactical and semantical understanding**
- “The topic of the lecture was interesting, but the lecturer was ???”
 - Grammatical form of predicted word must match; here we probably need an adjective
 - Model must realize that the next word characterizes the lecturer; not the lecture or its topic
 - Model must know what attributes are used to describe lecturers
 - Model must identify **but** as a negation such that the predicted word carries a sentiment which contrasts that of **interesting**
- **Language models rely on unsupervised learning and do not need labelled data**
 - Almost infinite amount of training data available; even better than in computer vision
 - Potentially facilitates pre-training for any language and not just English



Early forms of NLP transfer learning

Embeddings from Language Models (ELMo) and Universal Language Model Fine-Tuning for Text Classification (ULMFiT)

Considerations on NLP Transfer Learning

How to re-use pre-trained weights in downstream tasks?

■ Earlier studies identify two principled approaches

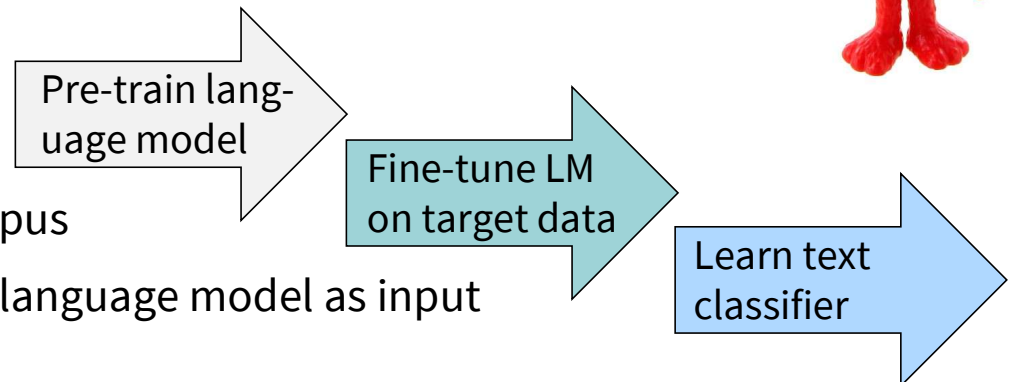
- Embeddings from language models (ELMo) by Peters et al. (2018)
- Universal language model fine tuning (ULMFiT) for text classification by Howard & Ruder (2018)

■ ELMo approach (feature extraction)

- Train a language model on a large corpus
- Add pre-trained representations as auxiliary features into a downstream model

■ ULMFiT approach (fine-tuning)

- Train a language model on a large corpus
- Fine-tune language model on the target task corpus
- Learn a target model, which uses the fine-tuned language model as input



Embeddings from Language Models (ELMo)

Deep contextualized word representations by Peters et al. (2018)

■ Word embedding desiderata

- Capture complex characteristics of word use (syntax and semantics)
- Capture how characteristics of word use vary across linguistic contexts

■ ELMo word vectors originate from a “**deep**” bidirectional language model (biLM)



Hey Elmo, what is the representation of the word “bank”?

Ähm, in which context?

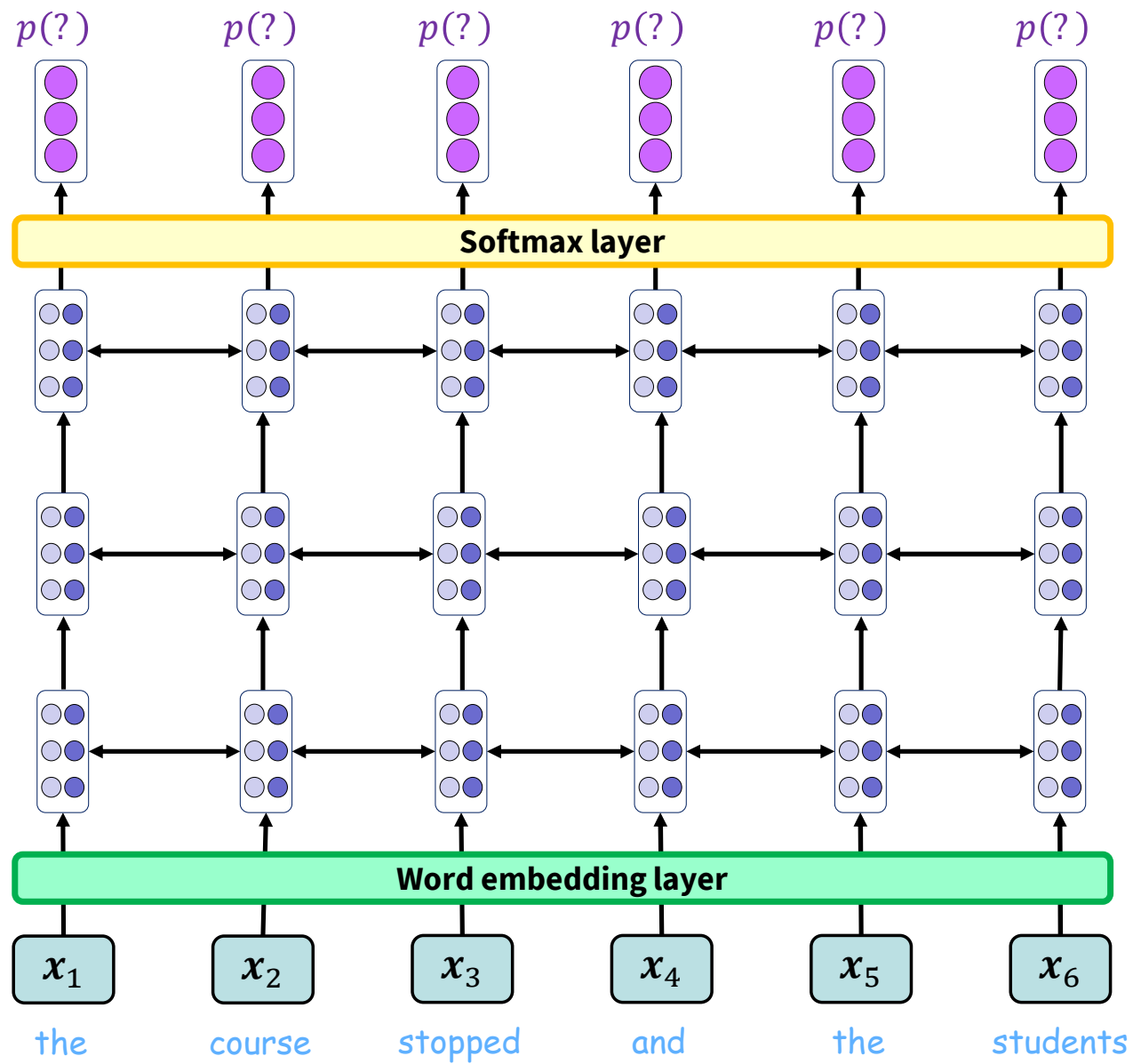
Well, „I'm your buddy, you can **bank** on me.”

Ok, then it is

1,82	-2,43	1,94	-0.93	...
------	-------	------	-------	-----



A Deep Bidirectional Language Model (biLM)



ELMo biLM Architecture

■ Bidirectional language model

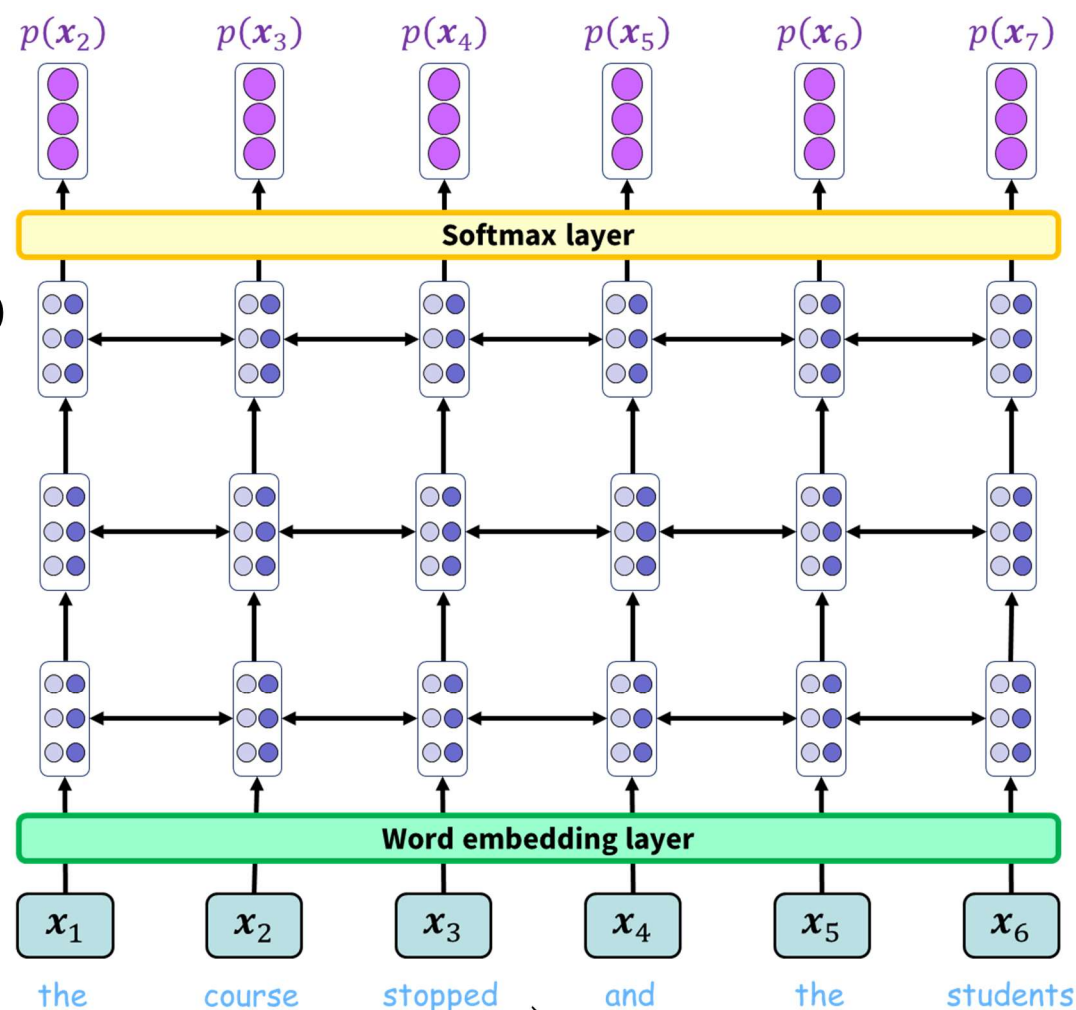
- Forward: $p(x_i, \dots, x_N) = \prod_{i=1}^N p(x_i | x_1, \dots, x_{i-1})$
- Backward: $p(x_i, \dots, x_N) = \prod_{i=1}^N p(x_i | x_{i+1}, \dots, x_N)$
- With sequence length N and x_i denoting a token

■ Stacked LSTM for both directions

- ELMo uses $L = 2$ layers with $d = 4096$ units
- Hidden state of token i at layer l : $\mathbf{h}_{il} = [\vec{\mathbf{h}}_{il}; \overleftarrow{\mathbf{h}}_{il}]$
- LSTM output $\mathbf{h}_{iL}$ predicts *next* token
- Forward & backward LSTM share embedding and softmax layer weights $\boldsymbol{\theta}_e$ and $\boldsymbol{\theta}_s$

■ Combined log-likelihood loss function

$$-\sum_{i=1}^n \left(\left(\log \left(p(x_i | x_1, \dots, x_{i-1}; \boldsymbol{\theta}_e, \vec{\boldsymbol{\theta}}_{LSTM}, \boldsymbol{\theta}_s) \right) \right) + \left(\log \left(p(x_i | x_{i+1}, \dots, x_N; \boldsymbol{\theta}_e, \overleftarrow{\boldsymbol{\theta}}_{LSTM}, \boldsymbol{\theta}_s) \right) \right) \right)$$



ELMo Representations

Novelty of ELMo comes from the way to obtain word representations

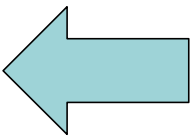
■ Represent a token x_i as a linear combination of corresponding hidden layers

□ L layer biLM gives $2L + 1$ representations per token x_i

□ $R_i = \{e_i, (\vec{h}_{il}; \overleftarrow{h}_{il} | l = 1, \dots, L)\} = \{h_{il} | l = 0, \dots, L\}$ where h_{i0} is the token layer (raw embedding)

■ Collapse all layers in R into a single vector for downstream model inclusion

$$\begin{aligned} & \text{ELMo}_i^{\text{task}} \\ & \parallel \\ & f(R_i, \Theta_{\text{task}}) = \gamma^{\text{task}} \times \sum \left\{ \begin{array}{ll} s_2^{\text{task}} \times \mathbf{h}_{i2} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \end{array} \\ s_1^{\text{task}} \times \mathbf{h}_{i1} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \end{array} \\ s_0^{\text{task}} \times \mathbf{h}_{i0} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \end{array} \end{array} \right. \end{aligned}$$



 $[\vec{h}_{il}; \overleftarrow{h}_{il}]$

The weights s_l^{task} are learned for each downstream task and normalized by softmax, while the factor γ^{task} allows the downstream model to scale the entire $\text{ELMo}_i^{\text{task}}$.

Application of ELMo in Downstream Tasks

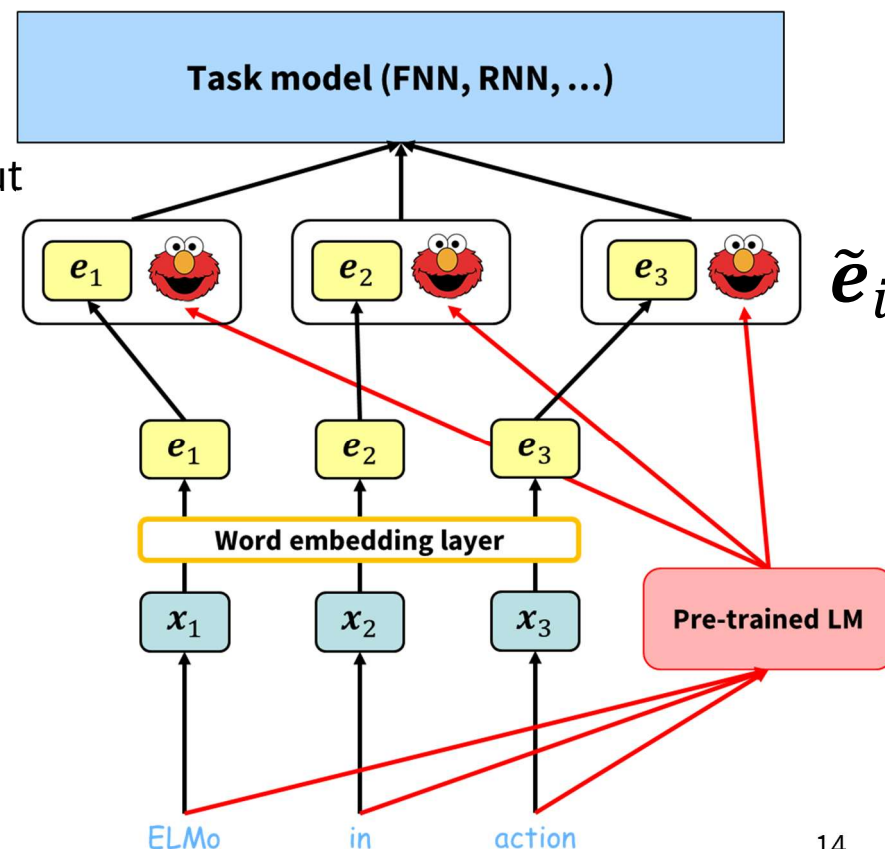
Embeddings are integrated into the input of the task-specific model

■ Vector concatenation

- Given target task token t_i with embedding e_i
- Construct $\tilde{e}_i = [e_i; \mathbf{ELMo}_i^{task}]$
- Train task model (arbitrary) on task corpus using $\tilde{e}_i$ as input

■ Contextualization

- ELMo uses bidirectional LSTMs, which account for the left and right context of words in the LM
- Auxiliary input $\mathbf{ELMo}_i^{task}$ to the task model is given as $f(\mathbf{R}_i, \Theta_{task}) = \gamma^{task} \sum_{l=0}^L s_l^{task} \mathbf{h}_{il}$
- Averaging of $\mathbf{h}_{il}$ is geared to the target task
- Weights s_l^{task} and scaling factor γ^{task} are tuned alongside other parameters of the task model
- Other LM parameter $\mathbf{R}_i$ are fixed and not altered when training the task model



The Importance of Depth

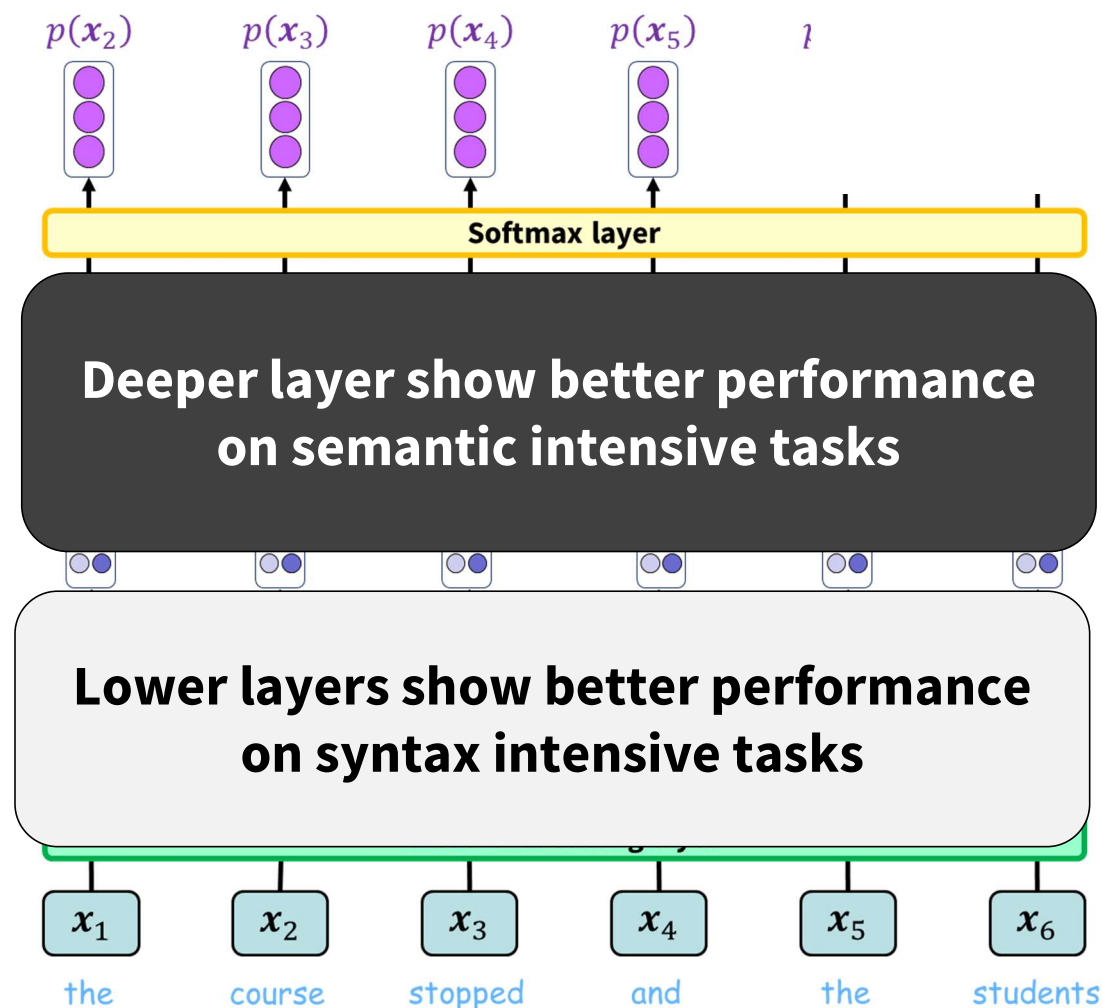
Different layers learn different things

■ Semantic intensive task

- Word sense disambiguation
 - “An apple a day keeps the doctor away”
 - “Just got my new iPhone. Apple rocks :)”
- Best performance with upper LSTM layer
- Similar with question answering, semantic role labeling, natural language inference

■ Syntax-intensive task

- POS Tagging
 - “Can you buy me a tea?”
 - MD PRP VB PRP DT NNP
- Best performance with lower LSTM layer
- Similar for named entity recognition



Universal Language Model Fine-Tuning for Text Classification (ULMFiT)

Howard & Ruder (2018)

■ Three stacked LSTM layers (unidirectional)

■ Pre-training as LM using general text

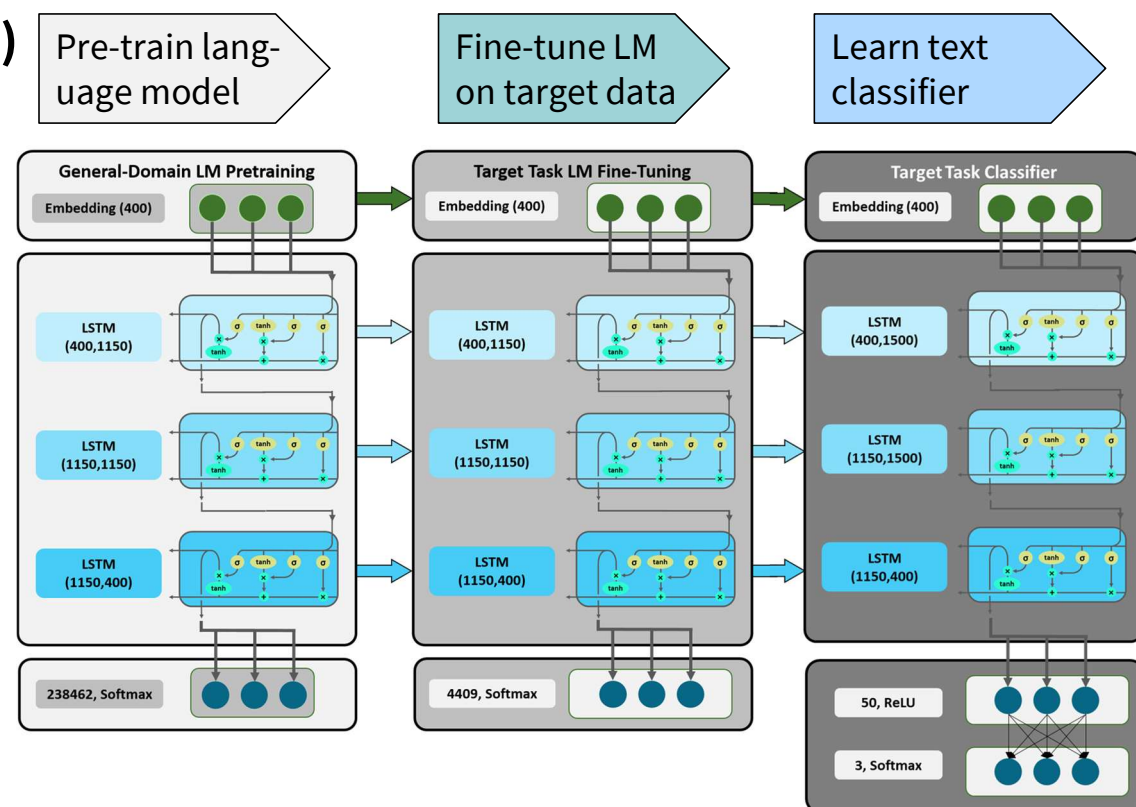
- 28,595 preprocessed Wikipedia articles
- 103 million words
- Word-level embedding with $d = 400$

■ Fine-tuning as LM using target corpus

- Analysis of vocabulary matching
- Address risk of *catastrophic forgetting*
 - Discriminative fine-tuning
 - Gradual unfreezing of layers
 - Dedicated learning rate schedules

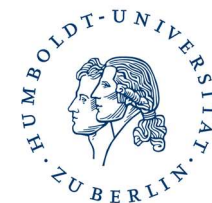
■ Fine-tune as classifier using target corpus

- Pool hidden states of the third LSTM layer
- Replace LM-Head with classification head (incl. dense layer)
- First train only classification layer, then gradually unfreeze other layers



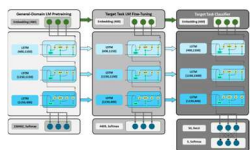
[Howard & Ruder (2018)]

2018 – The NLP ImageNet “Moment”



2018 – when NLP ImageNet became reality

ULMFiT



🦉 Jan 2018

🦿 1 GPU day

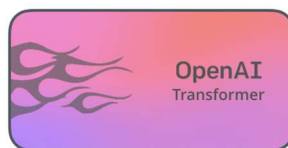
ELMo



🦉 Feb 2018

🦿 ~ 42 GPU days

GPT



🦉 Jun 2018

🦿 240 GPU days

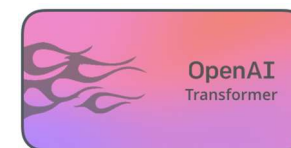
BERT



🦉 Oct 2018

🦿 256 TPU days

GPTv2



🦉 Feb 2019

🦿 ~2048 TPU days

fast.ai



 **OpenAI**

 **Google AI**

 **OpenAI**

2018 – The NLP ImageNet “Moment”

RNN-based language models dethroned

2018 – when NLP ImageNet became reality

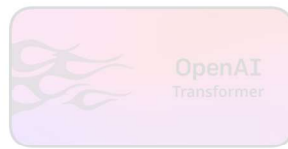
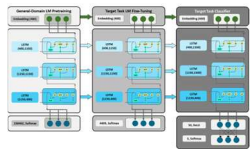
ULMFiT

ELMo

GPT

BERT

GPTv2



Jan 2018

Feb 2018

Jan 2018

Oct 2018

Feb 2019

1 GPU day

~ 42 GPU days

240 GPU days

256 TPU days

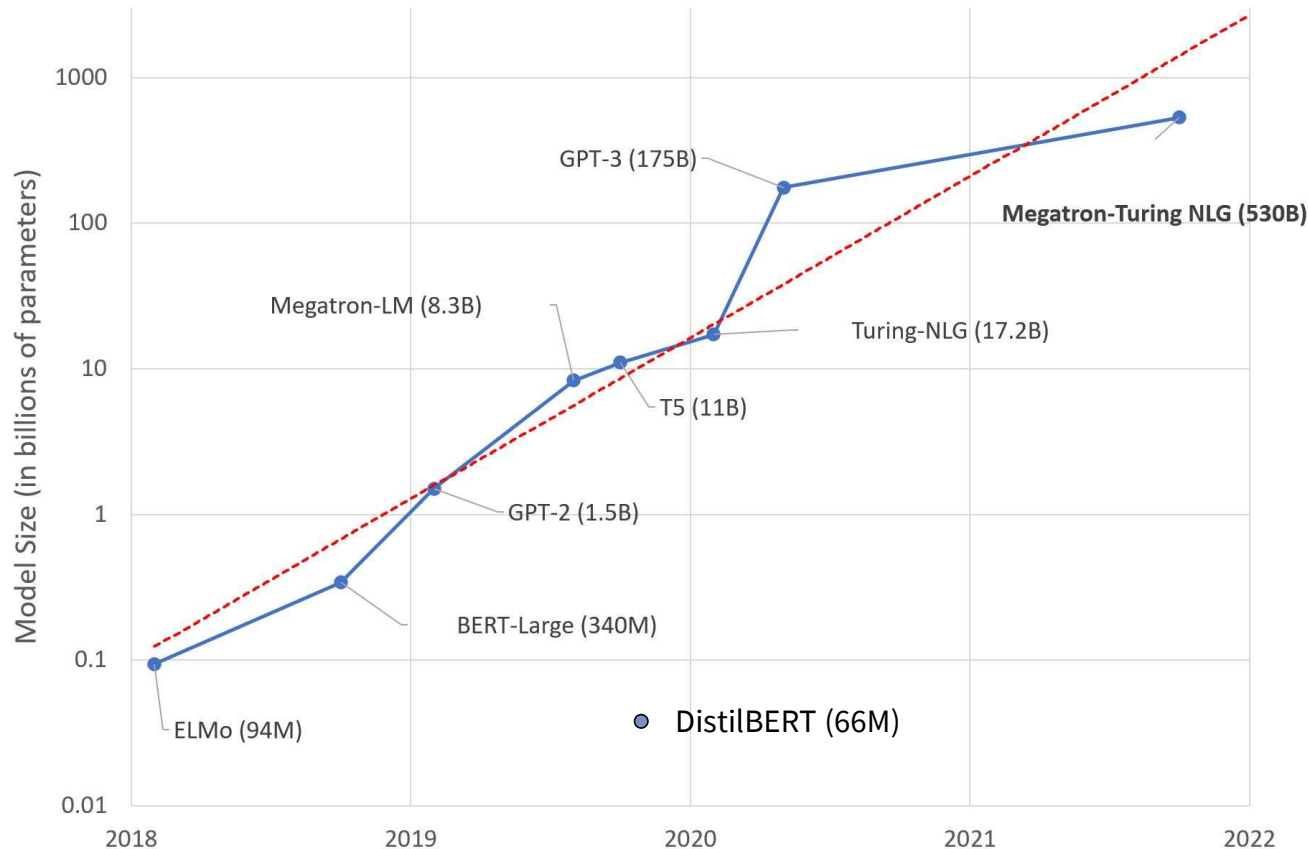
~2048 TPU days

Based on transformers



1 TPU day ~ 1.25 – 2.2 GPU days
according to Stanford lecture.

Trend Toward Larger Models Continues



Other mega models:

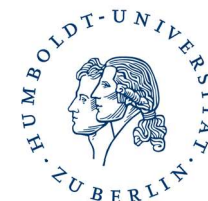
- PaLM, 540B (April 4 2022)
<https://ai.googleblog.com/2022/04/pathways-language-model-palm-scaling-to.html>
- Switch Transformer, up to 1.5T (2022)
<https://jmlr.org/papers/v23/21-0998.html>

And increasing criticism

- Environmental footprint
- Economic sanity
- See, e.g., Merity (2019) for a critical perspective on the benefits
- Strubell et al. (2019) and Dodge et al. (2022) touch on energy consumption

Image source: <https://www.microsoft.com/en-us/research/blog/using-deepspeed-and-megatron-to-train-megatron-turing-nlg-530b-the-worlds-largest-and-most-powerful-generative-language-model/>

The Rise of the Transformer



■ Sequence-to-sequence model architecture

- Principled architecture pattern for language-related tasks
- Traditionally implemented using RNNs

■ Attention mechanism

- Nature-inspired technique to overcome alignment issues in seq2seq tasks (e.g., machine translation)
- Seminal paper by Bahdanau et al. (2014) although around much longer
- Previously a component of RNN-based seq2seq models

■ Easily approachable descriptions

- Karim (2019)
- Several posts by Jay Alammar (<http://jalammar.github.io>)

Attention Is All You Need

NIPS 2017, pp. 6000-6010.

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
uszk@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

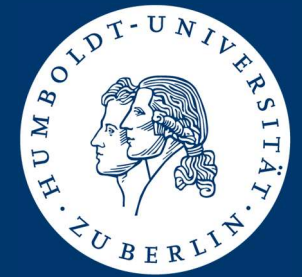
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions

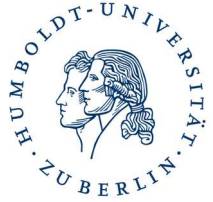
Annotated paper by Harvard NLP group
<http://nlp.seas.harvard.edu/2018/04/03/attention.html>



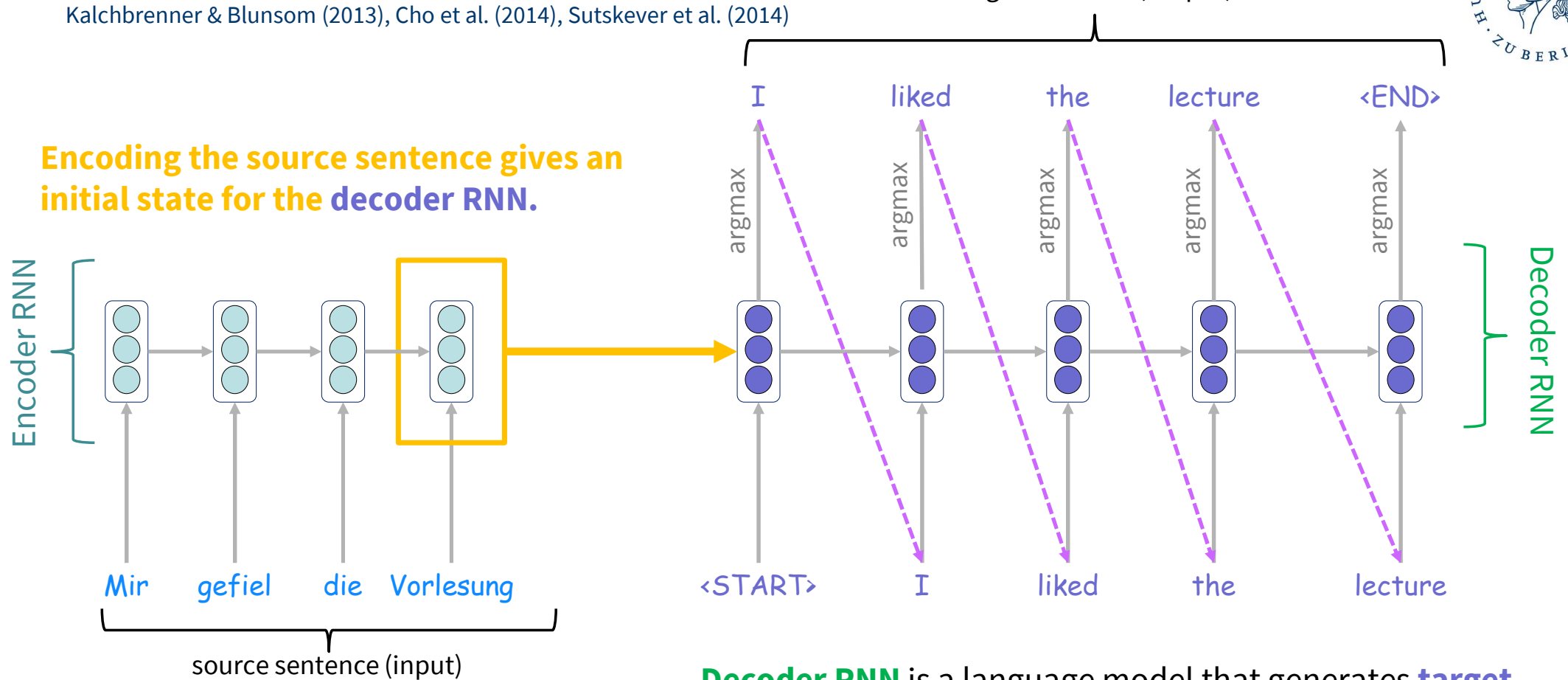
Attention Mechanism in Seq-to-Seq Architectures

Sequence-to-Sequence Architecture

Kalchbrenner & Blunsom (2013), Cho et al. (2014), Sutskever et al. (2014)



Encoding the source sentence gives an initial state for the decoder RNN.



Encoder RNN gives an encoding of the **source sentence**.

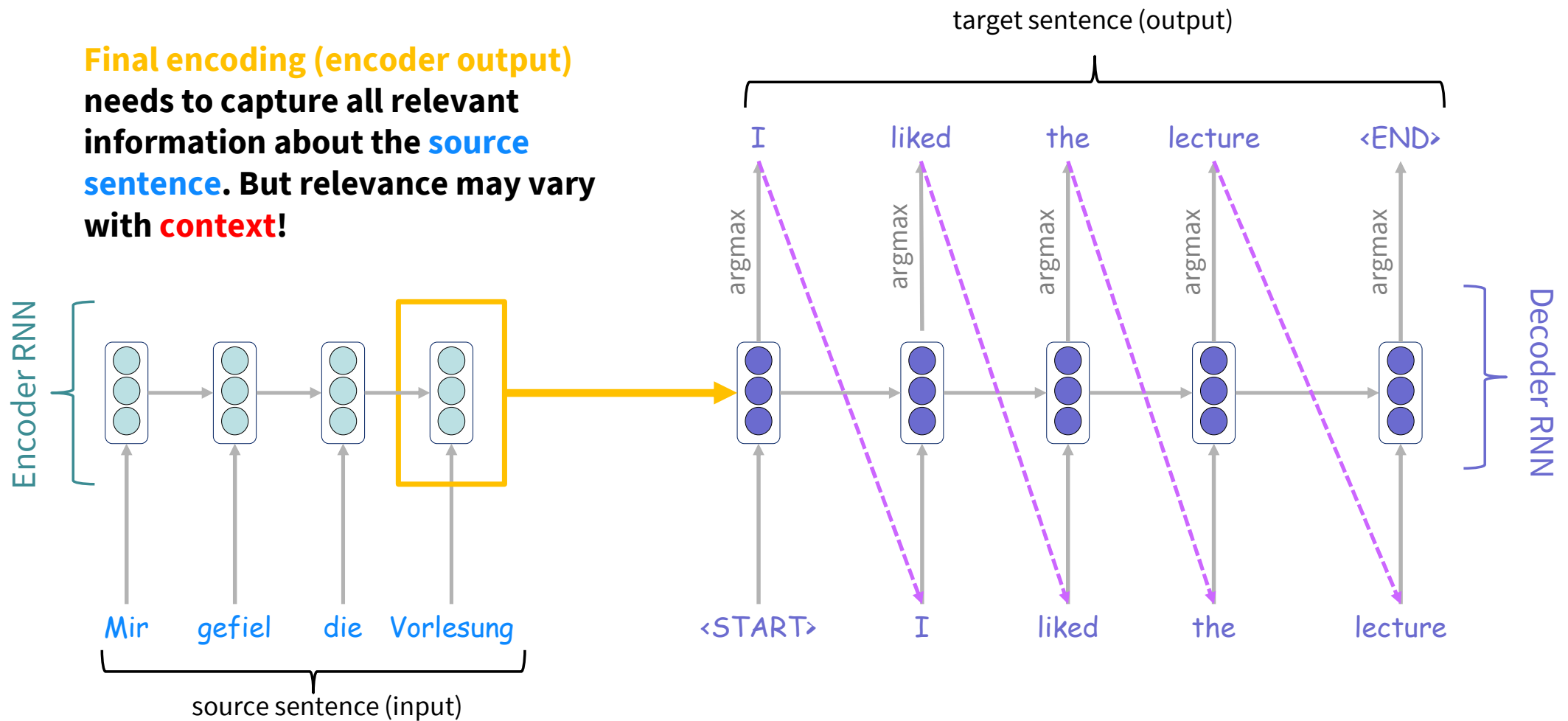
Decoder RNN is a language model that generates **target sentences conditioned on the encoding**.

$$P(\mathbf{y}|\mathbf{x}) = P(y_1|\mathbf{x})P(y_2|y_1, \mathbf{x})P(y_3|y_1, y_2, \mathbf{x}) \dots P(y_T|y_1 \dots y_{T-1}, \mathbf{x})$$

Attention Mechanism

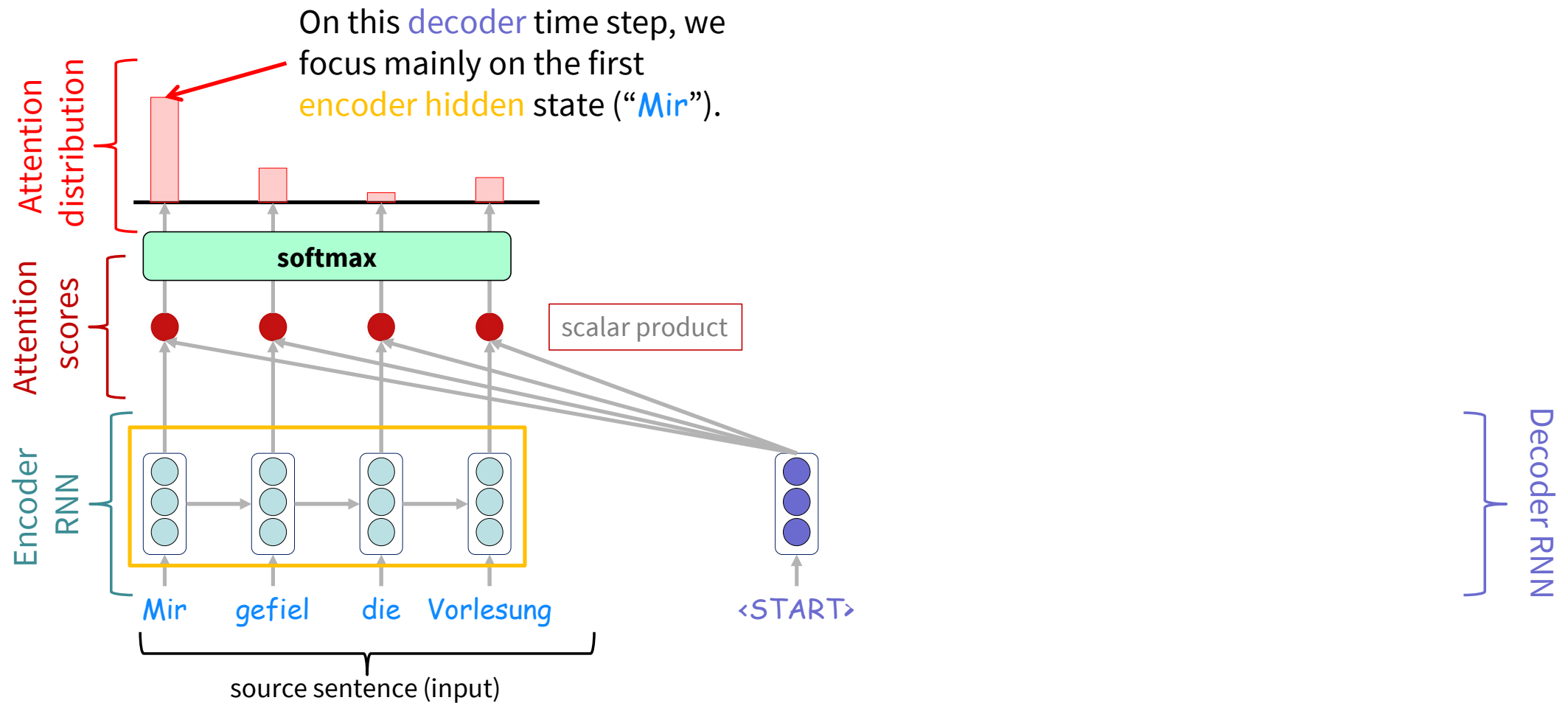
Overcoming information bottleneck

Final encoding (encoder output) needs to capture all relevant information about the **source sentence**. But relevance may vary with **context**!

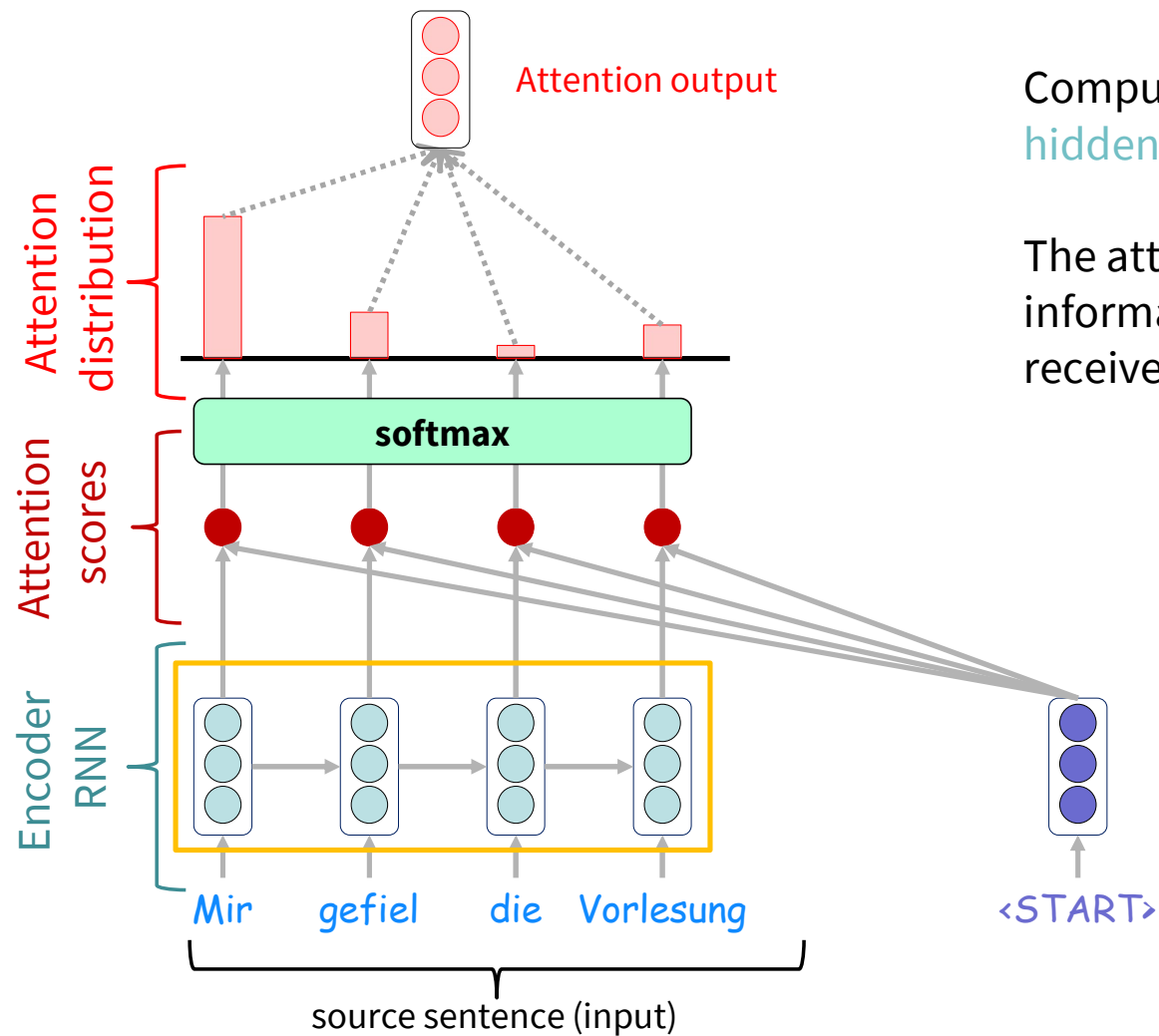


Attention Mechanism

Calculate attention distribution on each decoder step to recognize **context**



Attention Example in Neural Machine Translation

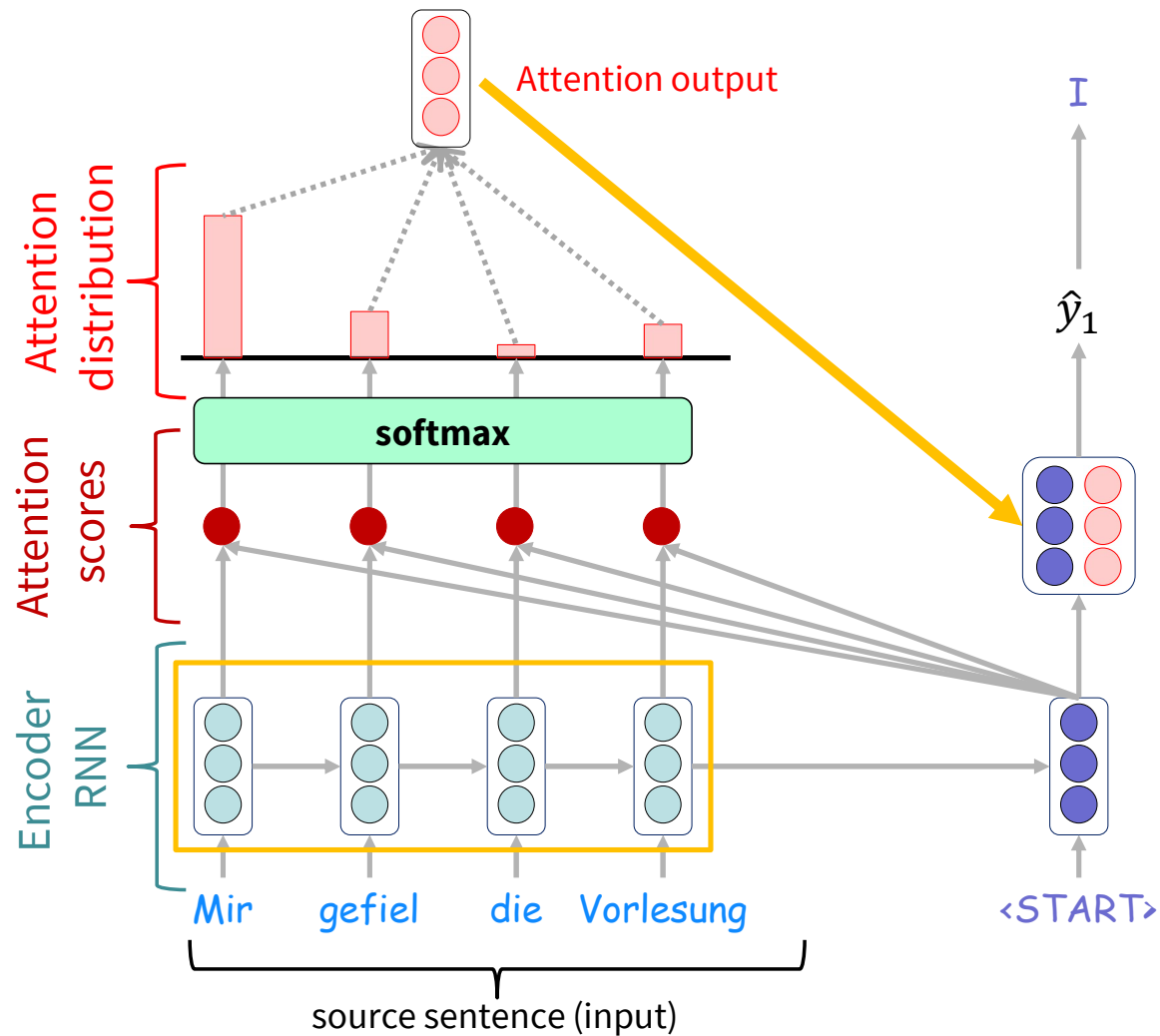


Compute a weighted sum of the **encoder hidden states** using the **attention distribution**.

The attention output is largely based on information from the hidden states that received **high attention**.

Decoder RNN

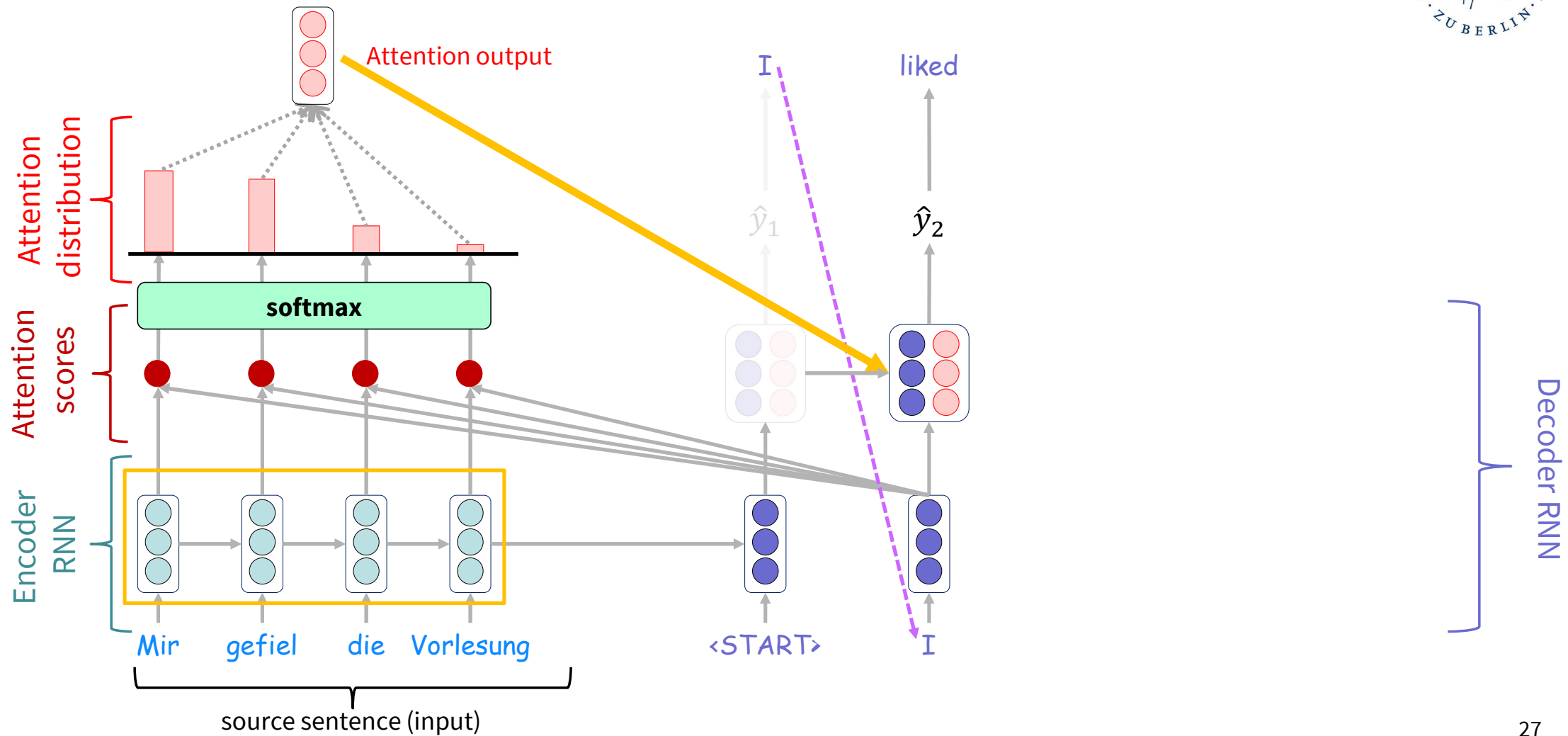
Attention Example in Neural Machine Translation



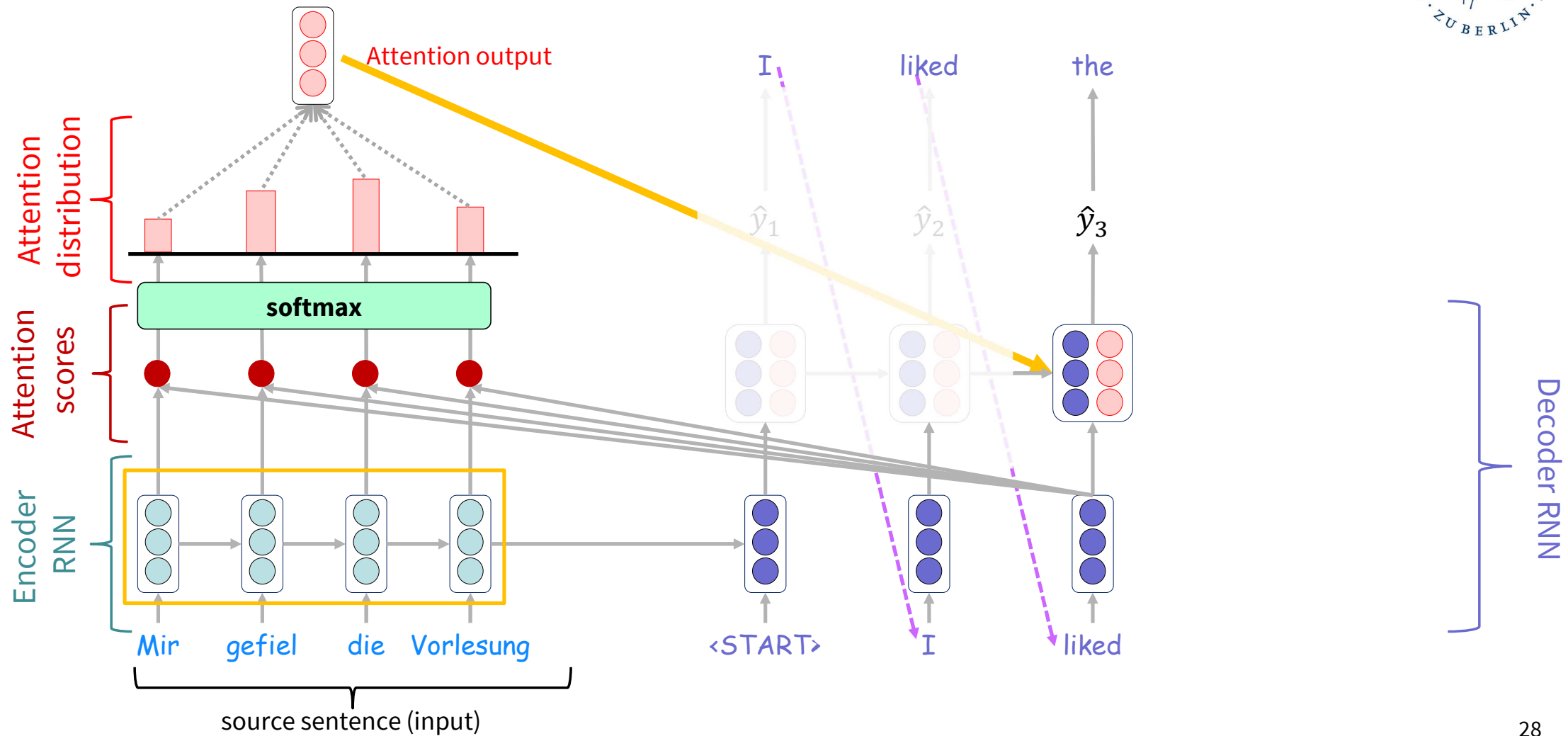
Combine **attention output** with **decoder hidden state**. Next, compute decoder output $\hat{y}$ as before.

Decoder RNN

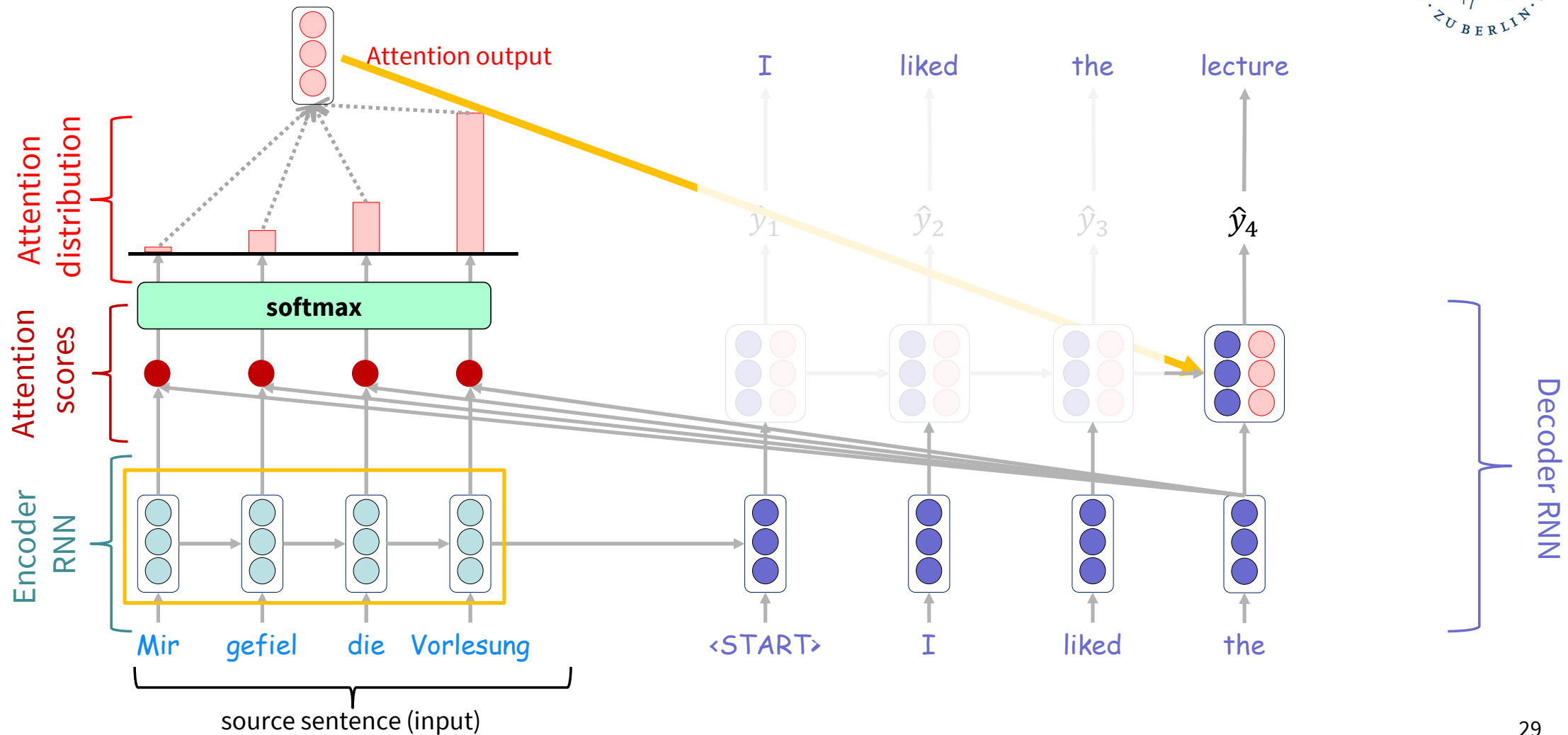
Attention Example in Neural Machine Translation



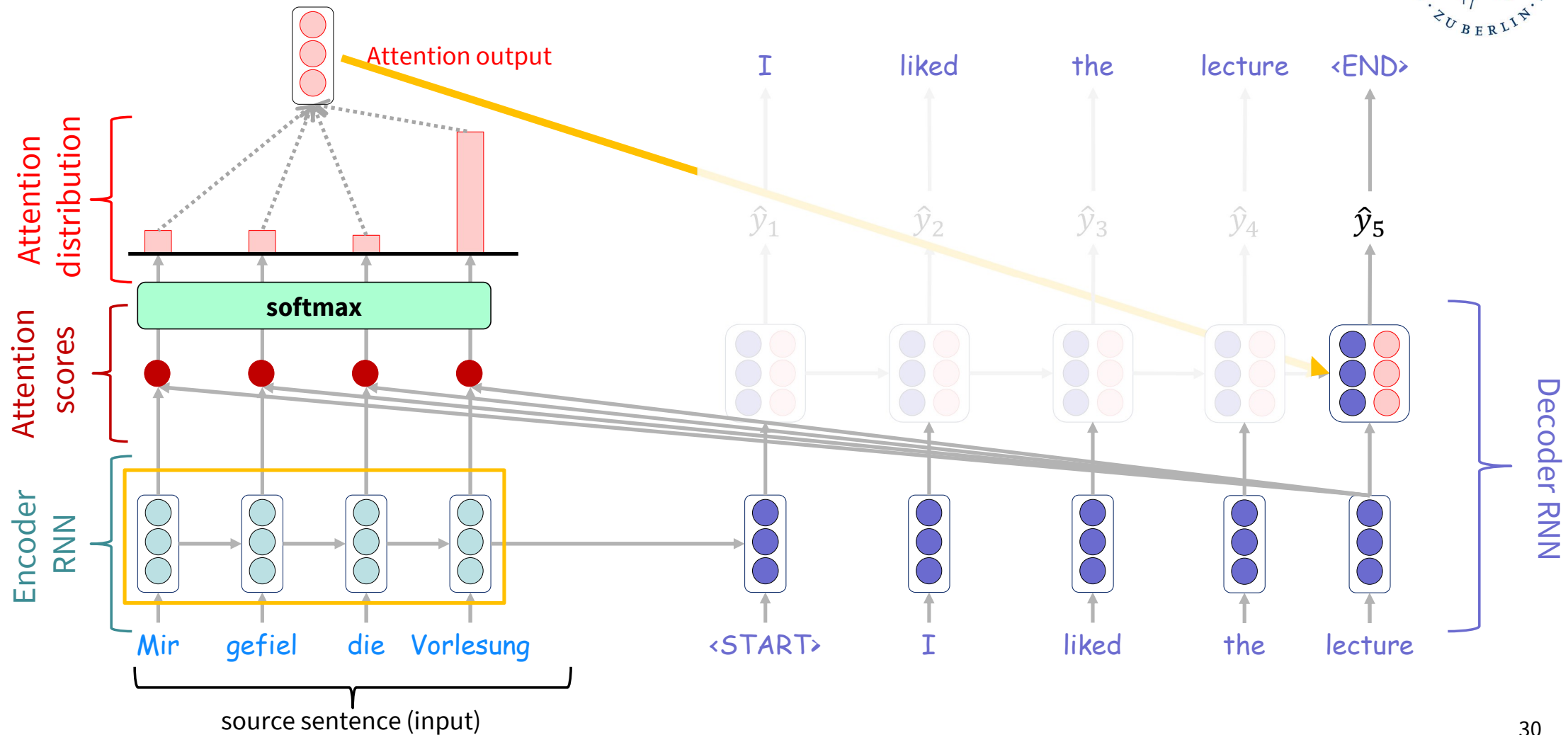
Attention Example in Neural Machine Translation



Attention Example in Neural Machine Translation



Attention Example in Neural Machine Translation



Attention Mechanism (Bahdanau et al. 2014)

Dynamically weighting input tokens using Query, Key, and Value

■ Formal characterization (Zhang et al. 2023)

- Let $\mathcal{D} \stackrel{\text{def}}{=} \{(\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_T, \mathbf{v}_T)\}$ denote a set of T tuples
- and $\mathbf{q}$ denote a query

$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{t=1}^T \alpha(\mathbf{q}, \mathbf{k}_t) \mathbf{v}_t$$

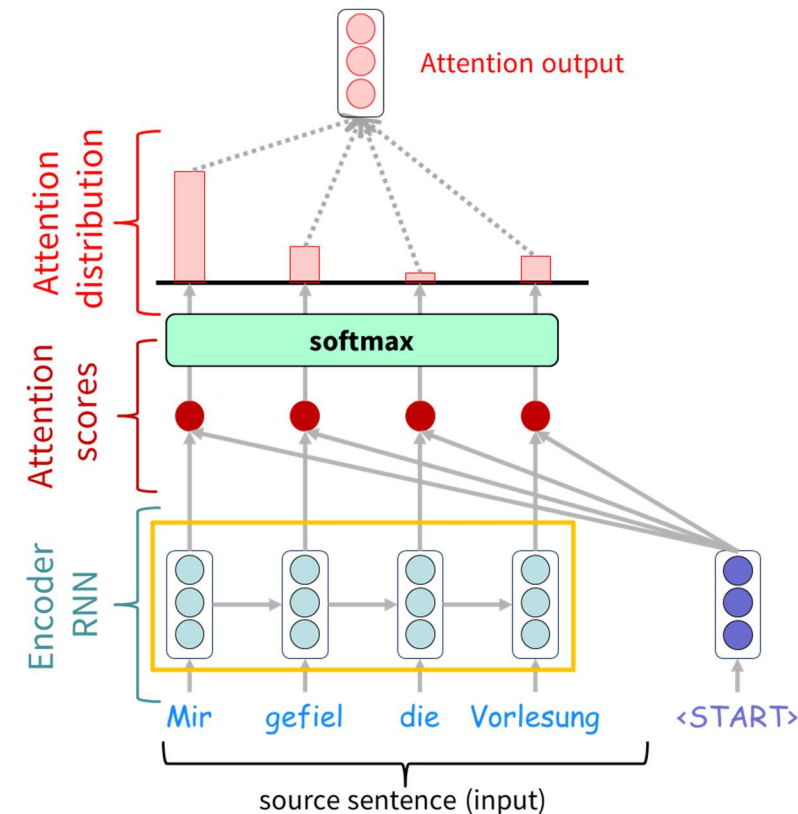
- where $\alpha(\mathbf{q}, \mathbf{k}_t) \in \mathbb{R} (t = 1, \dots, T)$ are scalar attention weights
- Attention acts like a “soft matching” of keys to the query
 - We compute a linear combination of values instead of retrieving just one
 - In a Python dictionary, for example, $\alpha(\mathbf{q}, \mathbf{k}_t)$ is exactly 1 for one t

■ Standard to have nonnegative weights that sum to unity

- Easily achieved using softmax

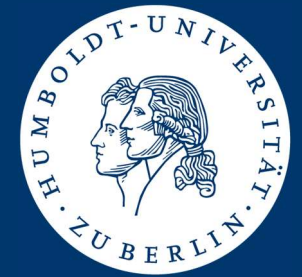
$$\alpha(\mathbf{q}, \mathbf{k}_j) = \frac{\exp(\alpha'(\mathbf{q}, \mathbf{k}_j))}{\sum_t (\alpha'(\mathbf{q}, \mathbf{k}_t))}$$

- Implies that we can use any function $\alpha'(\mathbf{q}, \mathbf{k}_t)$
- Several approaches/options (see, e.g., Zhang et al. (2023), Ch.11.2)



Summary of Attention Mechanism

- **Introduced in the scope of neural machine translation**
- **General deep learning mechanism compatible with various tasks and networks**
 - Feedforward, convolutional, recurrent networks
 - Text processing, image processing, structured data processing (seq2seq)
- **Definition:** given a set of **values** (vector) and a **query** (vector), attention computes a weighted sum over the values in which the weights depend on the query.
- **Intuition**
 - The weighted sum is a **selective summary** of the information contained in the **values**
 - The **query** determines which **values** are most relevant in the current context
- **Note**
 - Attention is also a way to obtain a fixed-size representation of an arbitrary **set of representations** dependent on some **other representation**
 - Recall pooling layers from the CNN chapter. Can think of attention as a ‘smarter’ way of pooling



Summary

Summary



Learning goals

- Considerations on how to carry out NLP transfer learning
- Early forms of NLP transfer learning
- Understand the attention mechanism



Findings

- Word embeddings facilitate shallow transfer learning
- Language modeling is an ideal task for pretraining
 - Requires syntactical and semantical text understanding
 - Unsupervised learning task benefiting from huge corpora
- Contextualization of word embeddings
- Finetuning runs the risk of catastrophic forgetting
- Attention is a sequence-to-sequence operation that computes outputs based on a selective summary of inputs



What next

- Transformer architecture
- Self-attention and how this is “all you need”

Literature



- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *ArXiv preprint*, arXiv:1409.0473.
- Cho, K., Merrienboer, B. v., Bahdanau, D., & Bengio, Y. (2014). *On the Properties of Neural Machine Translation: Encoder-Decoder Approaches*. In D. Wu, M. Carpuat, X. Carreras & E. M. Vecchi (Eds.). Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translatio, ACL, pp. 103-111.
- Howard, J., & Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. *CoRR*, arXiv:1801.06146.
- Kalchbrenner, N., & Blunsom, P. (2013). *Recurrent Continuous Translation Models*. Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, ACL, pp. 1700-1709.
- Merity, S. (2019). Single Headed Attention RNN: Stop Thinking With Your Head. *ArXiv preprint*, arXiv:1911.11423v2.
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *CoRR*, arXiv:1802.05365v2.
- Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and Policy Considerations for Deep Learning in NLP. Proceedings of the 57th Conference of the Association for Computational Linguistics (ACL'2019), Florence, Italy.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *ArXiv preprint*, arXiv:1409.3215v3.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NIPS'2017), Long Beach, CA, USA.
- Zeiler, M.D., & Fergus, R. (2013). Visualizing and Understanding Convolutional Networks. *ArXiv preprint*, 1311.2901.

Credits & Disclaimer

- **Course materials have been inspired and draw upon numerous great resources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources**
 - Colah's blog, Understanding LSTM networks (<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)
 - Zhang et al.: Dive into Deep Learning (<http://d2l.ai/index.html>)
 - MIT 6.S191: Deep Sequence Modeling (<http://introtodeeplearning.com/>)
 - Stanford University: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - Oxford University: Deep Natural Language Processing course (<https://github.com/oxford-cs-deepnlp-2017>)

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

